AgriAgent — Phase 2 and Phase 3: Progressive Demonstration & Evaluation Protocol

Two attempts will be given to each Team over the next few weeks. If you perform poorly, you can come back and redo it. However, your second attempt will also include Phase 3 evaluation.

Purpose and scope

Phase 2 is worth 40% of the project. It is scored out of 100 in a single 15- 20-minute live session, then scaled to 40. About 70 of the 100 points come from measured numbers that the team reports, and the evaluator rechecks them live. The rest comes from how well the team explains its design choices.

The evaluation session tests the **seven areas** in the Phase 2 instructions. It also includes the Phase 2 gate from the project methodology: live extraction on **3 unseen data points [any modality, including document, audio, language, etc.]**, an evaluation set that was not used in training, and a comparison with and without the adapter.

Three rules apply throughout:

- **Measured, not claimed.** A number counts only if the evaluator can re-run it.
- **Surprise inputs.** The instructor supplies the 3 unseen items during the session. This guards against agents that only work on cherry-picked documents.
- **Pending work earns credit when declared.** Sources or agents marked as pending in the mapping lose only coverage points. Anything claimed to work, but that fails in live conditions, loses its demonstration points as well.

Submission (GitHub, frozen before 22 October, if before the demo (marks deducted))

The deadline is 22 October, 23:59. The TA tags each team's last commit before the deadline as `phase2-submission`. The demo must run from that tag. Commits after the tag will be evaluated as overdue. The TA checks the items below **before** the session, so the session is spent on the demo rather than on missing files.

#	Required item	Path in repo	Pre-check
1	Source-to-agent map: every Phase 1 inventory source, its agent, and its status (done / partial / pending)	`docs/source_agent_map.csv`	Row count equals the Phase 1 inventory

#	Required item	Path in repo	Pre-check
2	Agent cards: sources, responsibility, inputs/outputs, model and tools, limitations	`docs/agents/<agent>.md`	One per agent
3	Challenge note with literature and tool tests	`docs/challenge.md`	At least 2 tools actually run, with results
4	Orchestration diagram and a one-command entry point	`README.md`, e.g. `python run.py --input <file>`	Runs on a clean clone
5	Evaluation set with checked reference outputs	`eval/cases/`, `eval/reference/`	Case counts meet the minimums below; no overlap with training data
6	Evaluation scripts (AgentEvals)	`eval/run_eval.py`	-----
7	Metrics report in the standard format	`eval/phase2_metrics.csv`	Template in the last section
8	Automation log: inputs attempted and manual interventions	`logs/automation_log.csv`	One row per input
9	Schema DDL, data dictionary, validation rules and a database dump	`db/schema.sql`, `db/dictionary.md`, `db/dump`	Loads without errors
10	Adapter comparison: base vs LoRA/QLoRA on the held-out set	`eval/adapter_comparison.csv`	Uses the same cases for both
11	Manifest: source, URL, collection date, language, license	`data/manifest.csv`	Every corpus file listed

Minimum evaluation set: at least 10 checked cases per implemented agent and at least 30 cases in total. At least 1 source group must be non-English (if language is taken as a challenge). A reference output counts as checked only if verified against the source

Demonstration protocol (15-20 minutes)

The session follows a fixed order. The surprise documents are handed over at minute 0 and run in the background, so their results are ready by the evaluation block. Each block has a time limit, and anything not shown within it is scored as not shown.

Time (min)	Block	What the team shows	What the evaluator does
0–2	Start	Clean clone at the `phase2-submission` tag;	Hands over 3 surprise items; the team starts the pipeline on them without editing any code
2–6	1. Mapping	Source-to-agent map, grouping rationale, pending sources	Picks 2 sources from the Phase 1 inventory and asks which agent handles them
15-20	2. Challenge	The challenge, what the literature covers, which tools were tested, and results	Asks whether a stated limitation is in the research or in the team's implementation
	3. Agents	For each implemented agent: source → decision → tool call → tool result → output, 1 hard or error case	Names which team member explains each agent; asks which decisions are made by the model and which by fixed code
	4. Orchestration	Multi-agent workflow; start, routing, hand-offs, completion	Asks for a **fault injection**:
	5. Evaluation	`run_eval.py` on the eval set; per-agent, per-group and end-to-end results; adapter comparison; surprise document results	The evaluator will check a sample of values against the source
	6–7. Automation and schema	Automation log; schema; 1 extracted record traced back to its source page	The evaluator will check a sample of values against the source
	Individual viva	—	1 role-specific question per member, checked against that member's commit history

Surprise items. The instructor picks the 3 documents from the team's own source types, for example, a mandi price PDF, a district crop report, or a scanned table, but from issues or districts that are not in the team's corpus. At least 1 is in the state's language. The instructor keeps a checked answer key of 10–15 fields per document. Teams may not edit code during the run. If they change configuration, for example, a file path, that **counts as a manual intervention**.

If the live run fails, the team may show a recorded run of the same code for half credit on that block. The failure itself goes in the automation log.

Definitions

Source group: Document, Audio, Database, or others

Agent: Domain-wise, multiple agents can be proposed

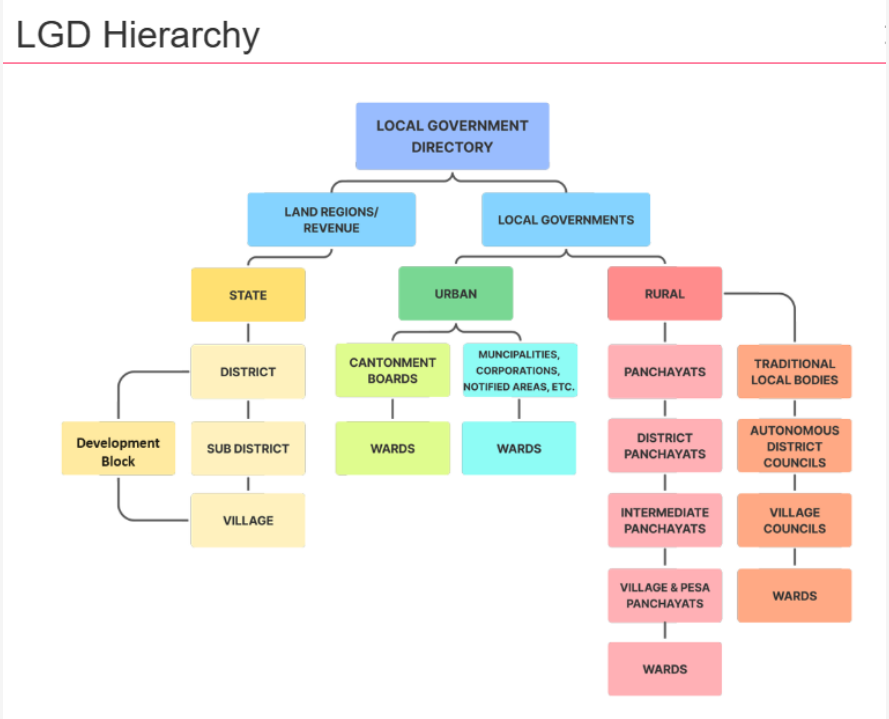
Quantitative metrics

The evaluation uses 18 metrics, each with a fixed definition, a floor, and a target. A score at or below the floor earns 0. A score at or above the target earns the full weight. Scores in between earn partial credit, scaled linearly:

Extraction and agent metrics are reported per agent and per source group, and the scored figure is the **macro-average across agents**. This way, one weak agent cannot hide behind a strong one that handles most of the volume.

Field match rule. An extracted field is correct only if all of the following hold: crop, location, and period match the reference after normalization; the value is exact for integers and codes, or within ±1% relative for decimals after converting units; and the unit is correct. A *correct number* but with a *wrong unit* counts as wrong. Each team declares its normalization, such as LGD codes for districts or crop synonym lists, in `eval/README.md` before the demo.

$$P = \frac{\text{correct fields extracted}}{\text{fields extracted}} \qquad R = \frac{\text{correct fields extracted}}{\text{fields in reference}} \qquad F_1 = \frac{2PR}{P+R}$$



Download codes for your state from here:
https://lgdirectory.gov.in/downloadDirectory.do?OWASP_CSRFTOKEN=WJWT-9M8E-LTVJ-6FSR-DUR3-GDPD-0IFE-VQMD

ID	Metric	Computed as	Tool/evidence	Floor	Target
M1	Source mapping coverage	Inventory sources mapped to an agent ÷ Phase 1 inventory sources	`source_agent_map.csv`	0.50	1.00
M2	Implementation coverage	Sources with an agent marked done or partial ÷ inventory sources	Same file, spot-checked live	0.20	0.60
M3	Field-level F1 (eval set)	F1 above on checked reference cases	Direct value check, script	0.50	0.85
M4	Provenance accuracy	Records whose page, table, row, URL or record ID leads to the value ÷ records sampled	Evaluator opens the source	0.60	0.95
M5	Surprise-item F1	F1 on the instructor's answer key for the 3 unseen items	Instructor key	0.30	0.75
M6	Tool selection accuracy	Correct tool chosen ÷ tool calls, compared with the reference trajectory	Foundry *Tool Selection*; AgentEvals	0.60	0.90
M7	Tool input accuracy	Calls with correct arguments (file, page range, language, schema) ÷ calls	Foundry *Tool Input Accuracy*	0.60	0.90
M8	Trajectory match	Cases whose trajectory matches the reference, with the match mode declared (strict, unordered, subset or superset) ÷ cases	AgentEvals trajectory match	0.50	0.80
M9	Task completion	Cases that produce a schema-valid final output with all required fields ÷ cases	Foundry *Task Completion*	0.50	0.90
M10	Groundedness	Output fields supported by the source text ÷ output fields	Foundry *Groundedness* or LLM judge, calibrated (below)	0.70	0.95
M11	End-to-end workflow success	Multi-agent runs reaching a correct "complete" status ÷	LangSmith traces	0.50	0.90

ID	Metric	Computed as	Tool/evidence	Floor	Target
		runs; plus 1 live fault injection handled (yes/no)			
M12	Automation rate	Inputs completed with no manual intervention ÷ inputs attempted	`automation_log.csv`	0.30	0.80
M13	Redundant call ratio (lower is better)	(Tool and LLM calls − calls in the reference trajectory) ÷ calls; retries reported separately	run stats	0.50	0.10
M14	Mandatory field completeness	Rows with source, source type, agent, run ID, locator, crop, location, time and unit filled, or explicitly coded as missing ÷ rows	SQL check	0.80	1.00
M15	Validation pass rate	Rows passing the declared rules (allowed units, valid LGD code, parsable date, value in plausible range) ÷ rows	SQL check	0.70	0.98
M16	Duplicate rate (lower is better)	Rows sharing a natural key (source, locator, crop, location, period, variable) ÷ rows	SQL `GROUP BY`	0.10	0.00
M17	Live trace-back	Random database rows correctly traced to their source location, out of 3	Evaluator picks rows	1	3
M18	Adapter comparison	F1(adapter) − F1(base) on the same held-out cases; overlap with training data = 0 by file hash	`adapter_comparison.csv`	See rubric	See rubric

Judge calibration (M6–M10). If an LLM judge produces a score, the team also reports the judge's agreement with human labels on at least 10 cases. If agreement is below 80%, the judge's score is replaced by the human-checked subset.

Reported but not scored: latency per document, tokens or cost per document, and retry count. These are a baseline for Phase 3.

Scoring rubric (100 points, scaled to 40)

Evaluation carries the most weight at 30 points, because it is where Phase 2 differs most from Phase 1. Of the 100 points, 69 are set by metrics or by binary evidence, 23 by judged explanation, and 8 by the individual viva.

Area	Pts	Measured part (pts)	Judged part (pts)
1. Sources, agents, responsibilities	8	M1 (3), M2 (2)	Grouping fits formats, languages, access methods (3)
2. Challenge and literature	8	At least 2 open-source tools actually run on the team's sources, with numbers reported (3)	What the literature covers; research vs implementation limits kept separate (5)
3. Individual agent demo	14	Share of implemented agents shown with a full source → decision → tool → result → output trace (6); a hard or error case shown live (3)	Clear about which decisions the model makes and which fixed code makes (5)
4. Orchestration	10	M11 success rate (4); fault injection handled (2)	Pattern justified against the sources; alternatives discussed (4)
5. Evaluation	30	M3 (6), M5 (6), M4 (3), M6–M10 (2 each, 10); M18: base and adapter on the same held-out cases (1), contamination check (1), result explained (1)	Evaluators chosen and skipped with reasons; judge calibrated (2)
6. Automation	10	M12 (6), M13 (2)	Manual steps and remaining barriers reported honestly (2)
7. Database schema	12	M14 (3), M15 (2), M16 (2), M17 (3)	Relationships, identifiers, missing-value handling explained (2)
Individual viva	8	—	Per member (see below)
Total	**100**	**69**	**31**

Judged parts use one 4-level scale, and the points awarded are the weight × level ÷ 3:

- **0:** absent.
- **1:** stated, but with no evidence.
- **2:** justified with evidence from the team's own sources or runs.
- **3:** level 2, plus alternatives compared with their trade-offs.

M18 does not reward a positive gain. An adapter that does not beat the base model scores full marks if the comparison is honest and explained. This follows the methodology's aim of measuring what fine-tuning adds.

Individual score = the team's 92 points + the member's own viva (out of 8). The viva question is about the member's declared role and the agent they built. If the member cannot explain code that their commit history attributes to them, they score 0 on the viva, and the evaluator flags it as a red flag.

Bonus

- Each additional evaluator that gives distinct, useful evidence, such as unit consistency or a cross-source agreement check: +1, maximum +2.
- A database populated with real extracted data from at least 2 source groups, with M17 = 3/3: +2.
- Non-English source group F1 reported separately and at or above the M3 target: +1.

Gate and red-flag rules (override the points):

Condition	Consequence
Surprise documents not attempted	Area 5 capped at 15; Phase 2 gate not passed until rerun
Evaluation cases found in the training data	M3 and M6–M10 score 0; a fresh held-out set is required before Phase 3
Code edited during the surprise run	That run counts as manual; M12 is computed accordingly

Metrics report template and logistics

Every team submits `eval/phase2_metrics.csv` in exactly this format. Because the format is the same for every team, the evaluator can sort and compare teams, and recompute the points from it. Teams use one row per metric per scope, where the scope is an agent name, a source group, *`end_to_end`* or `surprise`.

Please find the CSV file attached